

Quiero implementar en este repositorio (pablojimz/watch_gate) la parte consumidora de un flujo de sincronización de reglas Semgrep y YARA que vive en el repo privado pablojimz/Repo-reglas-SEMGREP-y-YARA. Ese repo, cuando publica una versión nueva de reglas (workflow `.github/workflows/release-rules.yml`, ya implementado y verificado en producción — releases v0.0.1 y v0.0.2 ya publicadas), dispara un evento `repository_dispatch` de tipo "rules-updated" con este payload real:

```
{
  "version": "v0.0.2",
  "generated_at": "2026-08-07T12:47:00Z",
  "hashes": {
    "semgrep": {
      "custom": {
        "python": "sha256:...",
        "dockerfile": "sha256:...",
        "...": "una entrada por cada carpeta de rules/semgrep/custom/ (17 lenguajes en total)"
      },
      "third_party": {
        "opengrep": { "python": "sha256:...", "generic": "sha256:..." },
        "trailofbits": { "python": "sha256:...", "rs": "sha256:..." },
        "elttam": { "go": "sha256:...", "yaml": "sha256:..." },
        "0xdea": { "c": "sha256:...", "noisy": "sha256:..." }
      }
    },
    "yara": {
      "antidebug_antivm": "sha256:...",
      "webshells": "sha256:..."
    }
  },
  "languages_changed": [],
  "third_party_changed": [],
  "yara_categories_changed": [],
  "rule_count": {
    "semgrep": { "custom": 64, "third_party": 717 },
    "yara": 695
  }
}
```

Estructura real del repo de reglas (IMPORTANTE — no es la carpeta plana SEMGREP/ que se planteó al principio del proyecto):

- `rules/semgrep/custom//` — reglas propias (17 lenguajes).
- `rules/semgrep/third-party///` — reglas de terceros integradas (vendors: `trailofbits`, `opengrep`, `elttam`, `0xdea`). no siempre es un nombre de lenguaje (existen "generic", "problem-based-packs", "noisy") y no siempre coincide con los nombres de `custom/` (p. ej. `trailofbits/rs` corresponde a `custom/rust`). Tráталos como namespaces independientes, no intentes mapear automáticamente de `third-party` a un lenguaje de `custom/`.

- rules/semgrep/config.yaml — NO es un fichero de reglas (es config de LiteLLM, sin relación); no lo incluyas en ningún checkout ni hash relacionado con reglas.
- dist/yara_scored// — artefacto YARA final publicado (sin cambios respecto a lo ya acordado: se aplican TODAS las categorías publicadas siempre, sin selección parcial).
- rules/semgrep/**/*.*.yaml está gestionado con Git LFS. Cualquier checkout de ese repo (parcial o completo) DEBE incluir lfs: true en el step de actions/checkout — sin eso, se descargan solo punteros LFS de 3 líneas en vez del contenido real de las reglas, lo cual rompería tanto el análisis como la verificación de hash de forma silenciosa (estarías hasheando el puntero, no la regla).

Importante sobre los hashes:

- El hash es POR CLAVE, nunca global. Para "semgrep.custom" es por lenguaje; para "semgrep.third_party" es por vendor y luego por carpeta (dos niveles de anidación); para "yara" es por categoría.
- Método determinista (igual en ambos repos, ya implementado en el repo de reglas en scripts/build_release_manifest.py): listar recursivamente los ficheros de la subcarpeta correspondiente, ordenarlos alfabéticamente por ruta relativa, concatenar su contenido binario sin separadores, calcular sha256 sobre esa concatenación. Replica exactamente este método (puedes pedirme el docstring de build_release_manifest.py si necesitas el detalle exacto).

Necesito dos workflows de GitHub Actions:

1. .github/workflows/sync-rules.yml — disparado por repository_dispatch (types: [rules-updated])
 - Hacer checkout del repo de reglas (pablojimz/Repo-reglas-SEMGREP-y-YARA) fijando **ref** exactamente al tag recibido en el payload (github.event.client_payload.version), NO a main. Incluye lfs: true en este checkout.
 - Usa sparse-checkout para traer:
 - a) rules/semgrep/custom// solo para los lenguajes en languages_changed
 - b) rules/semgrep/third-party/// solo para las entradas en third_party_changed (formato "/", parsear el string para construir la ruta)
 - c) TODAS las carpetas de dist/yara_scored// publicadas (no solo las de yara_categories_changed, porque el análisis YARA usa el catálogo completo)
 - d) el manifest.json completo, siempre
 - El checkout de ese repo privado necesita un token con permiso de lectura sobre él: léelo de un secret RULES_REPO_TOKEN.
 - VERIFICAR INTEGRIDAD POR CLAVE:
 - Para cada lenguaje en languages_changed: recalculer el hash de rules/semgrep/custom/ descargado y compararlo contra hashes.semgrep.custom..
 - Para cada entrada "/" en third_party_changed: recalculer el hash de rules/semgrep/third-party// descargado y compararlo contra hashes.semgrep.third_party...
 - Para cada categoría YARA descargada (todas las publicadas): recalculer el hash de dist/yara_scored/ y compararlo contra hashes.yara..
 - Usa en todos los casos el mismo método determinista descrito arriba.
 - Verifica cada clave de forma independiente y reporta en el log cuál falla si algo no coincide (nunca un resultado binario global "todo bien / todo mal").

- Si TODAS las claves verificadas coinciden: guardar la versión activa (por ejemplo en un fichero local `.rules-version` o como `artifact/cache`) y continuar.
 - Si alguna clave NO coincide: fallar el workflow explícitamente indicando qué clave(s) fallaron, con un mensaje claro tipo "Posible corrupción o manipulación de las reglas de — hash no coincide", y no activar esa versión para nada. No debe fallar en silencio.
 - Deja el resultado (versión activa + hashes verificados por clave) accesible para que los workflows de análisis de PRs lo puedan leer (por ejemplo como un `artifact` reutilizable, o comiteado en un fichero de estado dentro de este mismo repo).
2. `.github/workflows/reconcile-rules.yml` — disparado por `schedule` (cron diario, por ejemplo a las 03:00 UTC) como red de seguridad por si se pierde algún `repository_dispatch`
- Descargar SOLO `manifest.json` del repo de reglas (rama `main` o el último `release`), sin hacer `checkout` completo (no necesita `lfs: true`, `manifest.json` no está en LFS).
 - Comparar la `version` y todos los `hashes` (`semgrep.custom`, `semgrep.third_party`, `yara`) del `manifest` contra la versión/hashes actualmente activos en este repo.
 - Si difieren, ejecutar el mismo proceso de sincronización y verificación que el workflow 1 (puedes extraer la lógica común a una `action/script` reutilizable, incluyendo `lfs: true` para ese `checkout` completo).
 - Si tras la sincronización siguen sin coincidir, marcar el workflow como fallido para que quede visible.

Requisitos adicionales:

- El análisis de PRs debe usar la versión activa verificada tanto para `Semgrep` (`custom` + `third-party` del lenguaje modificado del PR) como para `YARA` (todas las categorías publicadas), sin descargar reglas por su cuenta sin pasar por esta verificación.
- Añade comentarios explicando cada paso, ya que esto forma parte de un proyecto académico de ciberseguridad y quiero poder justificar cada decisión de diseño (versionado + verificación de integridad por hash, separación `Semgrep custom/third-party/YARA`, manejo de `Git LFS`).
- El `secret DISPATCH_TOKEN` ya está configurado en el repo de reglas (`fine-grained PAT` sobre este repo, permiso `Contents: Read and write`) y verificado funcionando — no hace falta que lo gestiones tú, solo asume que el `repository_dispatch` va a llegar correctamente.